

Musterlösung Praktikum 9: Minimal aufspannende Bäume

1 Kruskal – AoC 2025 Day 8 „Playground“

Vollständige Lösung: `praktika/09_mst/aufgabe_kruskal.py`.

a) Modellierung

Bei n Junction Boxes gibt es

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

potenzielle Verbindungen. Die Lösung berechnet alle Paare, sortiert sie und speichert sie als Liste. Alternativ wäre ein **Min-Heap** möglich: alle Paare werden beim Start eingelegt, danach liefert `heappop()` die jeweils kürzeste Kante – ohne die gesamte Liste vorab zu sortieren. Der Heap ist auch für Teil 2 geeignet, da man einfach so lange `heappop()` aufruft, bis alle Boxen verbunden sind. Gegenüber dem sortierten Array bietet er keinen asymptotischen Vorteil ($O(n^2 \log n)$ in beiden Fällen), spart aber Speicher wenn – wie in Teil 1 – nur ein kleiner Teil der Kanten tatsächlich entnommen wird. Für typische AoC-Instanzen mit einigen hundert bis wenigen tausend Boxen ist das Sortieren aller Paare mit $O\left(\frac{n^2}{2} \log \frac{n^2}{2}\right)$ gut handhabbar.

b) Union-Find

```
class DisjointSet:

    def __init__(self):
        self._parent = {}
        self._size = {}

    def make_set(self, v):
        self._parent[v] = v
        self._size[v] = 1

    def find_set(self, v):
        if self._parent[v] != v:
            self._parent[v] = self.find_set(self._parent[v]) #
            Pfadkompression
        return self._parent[v]

    def union(self, u, v):
        ru, rv = self.find_set(u), self.find_set(v)
        if ru == rv:
            return False # bereits im selben Schaltkreis
        if self._size[ru] < self._size[rv]:
            ru, rv = rv, ru # Union by Size: kleinere Komponente
                             haengt sich an groessere
        self._parent[rv] = ru
        self._size[ru] += self._size[rv]
        return True
```

```

def size(self, v):
    return self._size[self.find_set(v)]

def circuit_sizes(self):
    return sorted(
        [self._size[v] for v in self._parent if self._parent[v] == v],
        reverse=True
    )

```

Erweiterungen gegenüber disjoint.py:

- `_size` verfolgt die Komponentengröße am jeweiligen Repräsentanten.
- *Union by Size*: Ohne diese Regel würde man stets die Wurzel der einen Komponente unter die Wurzel der anderen hängen – im schlimmsten Fall entsteht dabei eine lange Kette, weil wiederholt eine bereits große Komponente an eine kleine angehängt wird. Union by Size verhindert das, indem die kleinere Komponente immer an die größere angehängt wird. Damit kann ein Knoten höchstens dann tiefer im Baum wandern, wenn seine Komponente in eine mindestens gleich große eingebettet wird – was höchstens $\log n$ Mal hintereinander passieren kann. Der Baum bleibt also flach.
- *Pfadkompression* in `find_set`: `find_set` folgt dem Eltern-Zeiger rekursiv bis zur Wurzel. Ohne Optimierung bleibt diese Kette nach dem Aufruf unverändert; beim nächsten Aufruf für denselben oder einen Nachbarknoten muss derselbe Weg erneut durchlaufen werden. Pfadkompression hängt beim Rückgabeschritt alle besuchten Knoten direkt an die Wurzel um. Spätere `find_set`-Aufrufe auf denselben Knoten landen dann sofort bei der Wurzel, ohne den ursprünglichen Pfad nochmals traversieren zu müssen.

c) Kruskal

```

def solve(text, k=1000):
    boxes = parse_boxes(text)
    n = len(boxes)

    edges = sorted(
        ((euclidean(a, b), a, b) for a, b in combinations(boxes, 2)),
        key=lambda e: e[0]
    )

    ds = DisjointSet()
    for b in boxes:
        ds.make_set(b)

    # Teil 1: die k kuerzesten Paare verbinden (inkl. No-Ops)
    for _, u, v in edges[:k]:
        ds.union(u, v)
    idx = k

    sizes = ds.circuit_sizes()
    result1 = sizes[0] * sizes[1] * sizes[2]

    # Teil 2: weiter bis ein einziger Schaltkreis
    last_u = last_v = None
    while idx < len(edges):
        _, u, v = edges[idx]; idx += 1

```

```

        if ds.union(u, v):
            last_u, last_v = u, v
            if ds.size(last_u) == n:
                break

    result2 = last_u[0] * last_v[0]
    return result1, result2

```

Ausgabe für das Beispiel (20 Boxen, $k = 10$):

```

Anzahl Boxen: 20
Potenzielle Verbindungen: 190

--- Teil 1 (k=10) ---
Schaltkreisgroessen: [5, 4, 2, 2, 1, 1, 1, 1, 1, 1, 1]
Produkt der 3 groessten: 5 x 4 x 2 = 40

--- Teil 2 ---
Letzte Verbindung: (216, 146, 977) -- (117, 168, 530)
Produkt der X-Koordinaten: 216 x 117 = 25272

```

Aufruf:

```

python aufgabe_kruskal.py beispiel.txt 10      # Beispiel, k=10
python aufgabe_kruskal.py input.txt           # AoC-Input, k=1000

```

Wichtiger Unterschied zwischen Teil 1 und Teil 2: Teil 1 fordert, die k kürzesten Paare zu verbinden – unabhängig davon, ob die beiden Boxen bereits im selben Schaltkreis sind. Solche No-Op-Verbindungen reduzieren die Komponentenanzahl nicht. Teil 2 sagt hingegen explizit „closest *unconnected* pairs” – hier zählen nur erfolgreiche Unions, wie beim klassischen Kruskal.

Im Beispiel sind von den 10 kürzesten Paaren genau 9 erfolgreiche Verbindungen; das 10. Paar ist ein No-Op. Deshalb entstehen $20 - 9 = 11$ Schaltkreise (nicht 10).

d) AoC – Teil 1

Nach dem Verbinden der 1000 kürzesten Paare im persönlichen AoC-Datensatz liest man die drei größten Schaltkreisgrößen aus `circuit_sizes()[3]` und multipliziert sie. Das Ergebnis ist individuell je nach Eingabe.

e) AoC – Teil 2

Der eigene Algorithmus läuft nach Teil 1 einfach weiter, bis `ds.size(last_u) == n` gilt. Die letzte dabei hergestellte Verbindung liefert die gesuchten X-Koordinaten.

Warum ist dies gleichzeitig die teuerste Kante im MST? Kruskal fügt Kanten in aufsteigender Reihenfolge ein und überspringt Kanten, die einen Zyklus erzeugen würden. Die allerletzte Kante, die tatsächlich zwei Komponenten zusammenführt, ist per Konstruktion die schwerste Kante im entstehenden minimalen aufspannenden Baum.

Verifikation mit `graph.mst_kruskal()`: Die fertige Implementierung in `vorlesung/L08_graphen/graph.py` berechnet denselben MST. Da `mst_kruskal()` die Ergebniskanten intern nach Gewicht sortiert verarbeitet und in dieser Reihenfolge an `result` anhängt, ist `mst.edges[-1]` exakt die gesuchte letzte Verbindung.

```

g = AdjacencyListGraph()
for b in boxes:
    g.insert_vertex(f"{b[0]},{b[1]},{b[2]}")
for a, b in combinations(boxes, 2):
    d = euclidean(a, b)

```

```

g.connect(f"{a[0]},{a[1]},{a[2]}", f"{b[0]},{b[1]},{b[2]}", d)
g.connect(f"{b[0]},{b[1]},{b[2]}", f"{a[0]},{a[1]},{a[2]}", d)

mst_edges, _ = g.mst_kruskal()
last = mst_edges[-1]
gu = tuple(int(x) for x in last[0].split(','))
gv = tuple(int(x) for x in last[1].split(','))
assert gu[0] * gv[0] == result2    # muss mit eigener Loesung
    uebereinstimmen

```

Warum beide Richtungen eintragen? `connect()` ist gerichtet; `all_edges()` liefert nur explizit eingetragene Kanten. Für einen ungerichteten Graphen muss jedes Paar daher zweimal eingefügt werden. Der `same_set`-Check in `mst_kruskal()` filtert die Rückrichtung als No-Op heraus, so dass jede Kante im MST nur einmal landet.